

How does the @property decorator work in Python?

Asked 11 years, 5 months ago Modified 1 year, 1 month ago Viewed 828k times

I would like to understand how the built-in function `property` works. What confuses me is that `property` can also be used as a decorator, but it only takes arguments when used as a built-in function and not when used as a decorator.

This example is from the [documentation](#):

```
class C:
    def __init__(self):
        self._x = None

    def getx(self):
        return self._x
    def setx(self, value):
        self._x = value
    def delx(self):
        del self._x
    x = property(getx, setx, delx, "I'm the 'x' property.")
```

`property`'s arguments are `getx`, `setx`, `delx` and a doc string.

In the code below `property` is used as a decorator. The object of it is the `x` function, but in the code above there is no place for an object function in the arguments.

```
class C:
    def __init__(self):
        self._x = None

    @property
    def x(self):
        """I'm the 'x' property."""
        return self._x

    @x.setter
    def x(self, value):
        self._x = value

    @x.deleter
    def x(self):
        del self._x
```

How are the `x.setter` and `x.deleter` decorators created in this case?

[python](#) [properties](#) [decorator](#) [python-decorators](#) [python-internals](#) [Edit tags](#)

Share Edit Follow Close Flag Protect

edited Jan 13, 2021 at 23:00
user3064538

asked Jun 26, 2013 at 20:47
 [ashim](#)
25.5k 29 75 97

29 See also: [How do Python properties work?](#) – Martin Thoma Feb 3, 2015 at 8:24

8 `property` is actually a class (not a function), although it does probably does call the `__init__()` method when you make an object, of course. Using `help(property)` from the terminal is insightful. `help` is also a class for some reason. – Brötsyrfuzthrax Dec 8, 2016 at 8:17

4 I think this link provides a good example: [\[property\] \(journaldev.com/14893/python-property-decorator\)](#) – Sheng Bi Feb 7, 2019 at 16:55

9 @Shule 2-year-old thread, but still: **Everything** is a class. Even classes. – Miriam Apr 23, 2019 at 20:27

6 This was confusing to me too. I finally found an article that was able to break it down for me. I hope this helps someone else. [programiz.com/python-programming/property](#) I'm not affiliated in any way with the site. – jwdesign Jun 14, 2019 at 17:55

22 Answers

Sorted by: [Reset to default](#) Date modified (newest first)

The `property()` function returns a special [descriptor object](#):

```
>>> property()
<property object at 0x10ff07940>
```

It is this object that has *extra* methods:

```
>>> property().getter
<built-in method getter of property object at 0x10ff07998>
>>> property().setter
<built-in method setter of property object at 0x10ff07940>
>>> property().deleter
<built-in method deleter of property object at 0x10ff07998>
```

These act as decorators *too*. They return a new property object:

```
>>> property().getter(None)
<property object at 0x10ff079f0>
```

that is a copy of the old object, but with one of the functions replaced.

Remember, that the `@decorator` syntax is just syntactic sugar; the syntax:

```
@property
def foo(self): return self._foo
```

really means the same thing as

```
def foo(self): return self._foo
foo = property(foo)
```

so `foo` the function is replaced by `property(foo)`, which we saw above is a special object. Then when you use `@foo.setter()`, what you are doing is call that `property().setter` method I showed you above, which returns a new copy of the property, but this time with the setter function replaced with the decorated method.

The following sequence also creates a full-on property, by using those decorator methods.

First we create some functions:

```
>>> def getter(self): print('Get!')
...
>>> def setter(self, value): print('Set to {i}!'.format(value))
...
>>>
```

```
>>> def deleter(self): print('Delete!')
... 
```

Then, we create a `property` object with only a getter:

```
>>> prop = property(getter)
>>> prop.fget is getter
True
>>> prop.fset is None
True
>>> prop.fdel is None
True 
```

Next we use the `.setter()` method to add a setter:

```
>>> prop = prop.setter(setter)
>>> prop.fget is getter
True
>>> prop.fset is setter
True
>>> prop.fdel is None
True 
```

Last we add a deleter with the `.deleter()` method:

```
>>> prop = prop.deleter(deleter)
>>> prop.fget is getter
True
>>> prop.fset is setter
True
>>> prop.fdel is deleter
True 
```

Last but not least, the `property` object acts as a [descriptor object](#), so it has `__get__()`, `__set__()` and `__delete__()` methods to hook into instance attribute getting, setting and deleting:

```
>>> class Foo: pass
...
>>> prop.__get__(Foo(), Foo)
Get!
>>> prop.__set__(Foo(), 'bar')
Set to 'bar'!
>>> prop.__delete__(Foo())
Delete! 
```

The `Descriptor Howto` includes a [pure Python sample implementation](#) of the `property()` type:

```
class Property:
    "Emulate PyProperty_Type() in Objects/descrobject.c"

    def __init__(self, fget=None, fset=None, fdel=None, doc=None):
        self.fget = fget
        self.fset = fset
        self.fdel = fdel
        if doc is None and fget is not None:
            doc = fget.__doc__
        self.__doc__ = doc

    def __get__(self, obj, objtype=None):
        if obj is None:
            return self
        if self.fget is None:
            raise AttributeError("unreadable attribute")
        return self.fget(obj)

    def __set__(self, obj, value):
        if self.fset is None:
            raise AttributeError("can't set attribute")
        self.fset(obj, value)

    def __delete__(self, obj):
        if self.fdel is None:
            raise AttributeError("can't delete attribute")
        self.fdel(obj)

    def getter(self, fget):
        return type(self)(fget, self.fset, self.fdel, self.__doc__)

    def setter(self, fset):
        return type(self)(self.fget, fset, self.fdel, self.__doc__)

    def deleter(self, fdel):
        return type(self)(self.fget, self.fset, fdel, self.__doc__)
```

Share Edit Follow Flag

edited Oct 18, 2023 at 1:02

answered Jun 26, 2013 at 20:54

 [Mateen Ulhaq](#)

27.1k 21 117 152



[Martijn Pieters](#)

1.1m 318 4.2k 3.4k

16  Very good. You could add the fact that after `Foo.prop = prop` you can do `Foo().prop = 5`; `pront Foo().prop`; `del Foo().prop` with the desired outcome.
– [gligigi](#) Jun 27, 2013 at 7:25

16  Method objects are created on the fly and *can* reuse the same memory location if available. – [Martijn Pieters](#) Jun 12, 2014 at 19:15

4  @MarkusMeskanen: I rather use `type()` as accessing dunder attributes and methods are meant to be used as extension points by the standard functions and operators. – [Martijn Pieters](#) Mar 10, 2015 at 21:46

5  @MarkusMeskanen: because the object is immutable, and if you mutated it in place you could not specialise it in a subclass. – [Martijn Pieters](#) Jun 13, 2015 at 14:25

12  @MarkusMeskanen: see [Python overriding getter without setter](#); if `@human.name.getter` altered the `property` object in-place rather than return a new, the `human.name` attribute would be altered, changing the behaviour of that superclass. – [Martijn Pieters](#) Jun 13, 2015 at 14:33

|



Let's start with Python decorators.

51

A Python decorator is a function that helps to add some additional functionalities to an already defined function.



In Python, everything is an object. Functions in Python are first-class objects which means that they can be referenced by a variable, added in the lists, passed as arguments to another function, etc.



Consider the following code snippet.



```
def decorator_func(fun):
    def wrapper_func():
        print("Wrapper function started")
        fun()
        print("Given function decorated")
        # Wrapper function add something to the passed function and decorator
        # returns the wrapper function
    return wrapper_func

def say_bye():
    print("bye!!")
```

```

say_bye = decorator_func(say_bye)
say_bye()

# Output:
# Wrapper function started
# bye!!
# Given function decorated

```

Here, we can say that the decorator function modified our `say_bye` function and added some extra lines of code to it.

Python syntax for decorator

```

def decorator_func(fun):
    def wrapper_func():
        print("Wrapper function started")
        fun()
        print("Given function decorated")
        # Wrapper function add something to the passed function and decorator
        # returns the wrapper function
    return wrapper_func

@decorator_func
def say_bye():
    print("bye!!!")

say_bye()

```

Let's go through everything with a case scenario. But before that, let's talk about some OOP principles.

Getters and setters are used in many object-oriented programming languages to ensure the principle of data encapsulation(which is seen as the bundling of data with the methods that operate on these data.)

These methods are, of course, the getter for retrieving the data and the setter for changing the data.

According to this principle, the attributes of a class are made private to hide and protect them from other code.

Yup, **@property** is basically a *pythonic way to use getters and setters*.

Python has a great concept called property which makes the life of an object-oriented programmer much simpler.

Let us assume that you decide to make a class that could store the temperature in degrees Celsius.

```

class Celsius:
    def __init__(self, temperature = 0):
        self.set_temperature(temperature)

    def to_fahrenheit(self):
        return (self.get_temperature() * 1.8) + 32

    def get_temperature(self):
        return self._temperature

    def set_temperature(self, value):
        if value < -273:
            raise ValueError("Temperature below -273 is not possible")
        self._temperature = value

```

Refactored Code, Here is how we could have achieved it with 'property.'

In Python, `property()` is a built-in function that creates and returns a property object.

A property object has three methods, `getter()`, `setter()`, and `delete()`.

```

class Celsius:
    def __init__(self, temperature = 0):
        self.temperature = temperature

    def to_fahrenheit(self):
        return (self.temperature * 1.8) + 32

    def get_temperature(self):
        print("Getting value")
        return self.temperature

    def set_temperature(self, value):
        if value < -273:
            raise ValueError("Temperature below -273 is not possible")
        print("Setting value")
        self.temperature = value

temperature = property(get_temperature, set_temperature)

```

Here,

```

temperature = property(get_temperature, set_temperature)

```

could have been broken down as,

```

# make empty property
temperature = property()
# assign fget
temperature = temperature.getter(get_temperature)
# assign fset
temperature = temperature.setter(set_temperature)

```

Point To Note:

- `get_temperature` remains a property instead of a method.

Now you can access the value of temperature by writing.

```

C = Celsius()
C.temperature
# instead of writing C.get_temperature()

```

We can go on further and not define names `get_temperature` and `set_temperature` as they are unnecessary and pollute the class namespace.

The **pythonic way** to deal with the above problem is to use **@property**.

```

class Celsius:
    def __init__(self, temperature = 0):
        self.temperature = temperature

    def to_fahrenheit(self):
        return (self.temperature * 1.8) + 32

    @property
    def temperature(self):
        print("Getting value")
        return self._temperature

    @temperature.setter
    def temperature(self, value):
        if value < -273:

```

```

        raise ValueError("Temperature below -273 is not possible")
    print("Setting value")
    self.temperature = value

```

Points to Note -

1. A method that is used for getting a value is decorated with "@property".
2. The method which has to function as the setter is decorated with "@temperature.setter", If the function had been called "x", we would have to decorate it with "@x.setter".
3. We wrote "two" methods with the same name and a different number of parameters, "def temperature(self)" and "def temperature(self,x)".

As you can see, the code is definitely less elegant.

Now, let's talk about one real-life practical scenario.

Let's say you have designed a class as follows:

```

class OurClass:
    def __init__(self, a):
        self.x = a

y = OurClass(10)
print(y.x)

```

Now, let's further assume that our class got popular among clients and they started using it in their programs. They did all kinds of assignments to the object.

And one fateful day, a trusted client came to us and suggested that "x" has to be a value between 0 and 1000; this is really a horrible scenario!

Due to properties, it's easy: We create a property version of "x".

```

class OurClass:
    def __init__(self, x):
        self.x = x

    @property
    def x(self):
        return self.__x

    @x.setter
    def x(self, x):
        if x < 0:
            self.__x = 0
        elif x > 1000:
            self.__x = 1000
        else:
            self.__x = x

```

This is great, isn't it? You can start with the simplest implementation imaginable, and you are free to later migrate to a property version without having to change the interface! So properties are not just a replacement for getters and setters!

You can check this Implementation [here](#)

Share Edit Follow Flag

edited May 11, 2023 at 21:58

 InSync
10.1k 4 13 49

answered Sep 17, 2018 at 0:01

 Divyanshu Rawat
4,681 3 41 59

7  Your Celsius class is going to infinitely recurse when setting (which means upon instantiation). – Ted Petrou Sep 23, 2018 at 12:25

1  @Ted Petrou I Didn't get you? How it will infinitely recurse when setting? – Divyanshu Rawat Sep 23, 2018 at 12:37 

1  People are asking how does it work not why it works? @ShengBi – Divyanshu Rawat Feb 7, 2019 at 17:53 

2  compared to the top voted answers, this one is designed for humans; thanks. – InfoSek May 18, 2020 at 0:06

2  You should make the change of both temperature getter and setter method from self.temperature to self._temperature, else it's running recursively. – Ang Yiwei Sep 27, 2021 at 8:13

|

 Below is another example on how @property can help when one has to refactor code which is taken from [here](#) (I only summarize it below):

123 Imagine you created a class Money like this:

```

class Money:
    def __init__(self, dollars, cents):
        self.dollars = dollars
        self.cents = cents

```

and a user creates a library depending on this class where he/she uses e.g.

```

money = Money(27, 12)

print("I have {} dollar and {} cents.".format(money.dollars, money.cents))
# prints I have 27 dollar and 12 cents.

```

Now let's suppose you decide to change your Money class and get rid of the dollars and cents attributes but instead decide to only track the total amount of cents:

```

class Money:
    def __init__(self, dollars, cents):
        self.total_cents = dollars * 100 + cents

```

If the above mentioned user now tries to run his/her library as before

```

money = Money(27, 12)

print("I have {} dollar and {} cents.".format(money.dollars, money.cents))

```

it will result in an error

```

AttributeError: 'Money' object has no attribute 'dollars'

```

That means that now everyone who relies on your original Money class would have to change all lines of code where dollars and cents are used which can be very painful... So, how could this be avoided? By using @property !

That is how:

```
class Money:
    def __init__(self, dollars, cents):
        self.total_cents = dollars * 100 + cents

    # Getter and setter for dollars...
    @property
    def dollars(self):
        return self.total_cents // 100

    @dollars.setter
    def dollars(self, new_dollars):
        self.total_cents = 100 * new_dollars + self.cents

    # And the getter and setter for cents.
    @property
    def cents(self):
        return self.total_cents % 100

    @cents.setter
    def cents(self, new_cents):
        self.total_cents = 100 * self.dollars + new_cents
```

when we now call from our library

```
money = Money(27, 12)

print("I have {} dollar and {} cents.".format(money.dollars, money.cents))
# prints I have 27 dollar and 12 cents.
```

it will work as expected and we did not have to change a single line of code in our library! In fact, we would not even have to know that the library we depend on changed.

Also the setter works fine:

```
money.dollars += 2
print("I have {} dollar and {} cents.".format(money.dollars, money.cents))
# prints I have 29 dollar and 12 cents.

money.cents += 10
print("I have {} dollar and {} cents.".format(money.dollars, money.cents))
# prints I have 29 dollar and 22 cents.
```

You can use `@property` also in abstract classes; I give a minimal example [here](#).

Share Edit Follow Flag

edited Dec 27, 2022 at 0:30

answered Sep 23, 2018 at 11:58

 **starball**
48k 28 182 839

 **Cleb**
26k 23 123 161

- 8 ▲ your summary is very good, the example that website takes is a little bit strange .. A beginner would ask .. why can't we just stick to the `self.dollar = dollars` ? we have done so much with `@property`, but it seems no extract functionality is added. – Sheng Bi Feb 7, 2019 at 16:39
- 6 ▲ @ShengBi: Do not focus that much on the actual example but more on the underlying principle: If - for what ever reason - you have to refactor code, you can do so without affecting anyone other's code. – Cleb Feb 7, 2019 at 16:44
- 4 ▲ @cleb you da real mvp. Everybody else uses that getter setter example like this one, programiz.com/python-programming/property. But you are the only one that actually explains why we want property. It's because when we writes something upon which a lot of people are going to build, we want to be able to modify the base classes without no real impact on how the successors use or build upon our work, implementation wise. – Chen Lizi Apr 7, 2021 at 0:28



In the following, I have given an example to clarify `@property`

2

Consider a class named `Student` with two variables: `name` and `class_number` and you want `class_number` to be in the **range of 1 to 5**.



Now I will explain two wrong solutions and finally the correct one:



The code below is **wrong** because it doesn't validate the `class_number` (to be in the range 1 to 5)



```
class Student:
    def __init__(self, name, class_number):
        self.name = name
        self.class_number = class_number
```

Despite validation, this solution is also **wrong**:

```
def validate_class_number(number):
    if 1 <= number <= 5:
        return number
    else:
        raise Exception("class number should be in the range of 1 to 5")

class Student:
    def __init__(self, name, class_number):
        self.name = name
        self.class_number = validate_class_number(class_number)
```

Because `class_number` validation is checked only at the time of making a class instance and it is not checked after that (it is possible to change `class_number` with a number outside of the range 1 to 5):

```
student1 = Student("masoud", 5)
student1.class_number = 7
```

The **correct** solution is:

```
class Student:
    def __init__(self, name, class_number):
        self.name = name
        self.class_number = class_number

    @property
    def class_number(self):
        return self._class_number

    @class_number.setter
    def class_number(self, class_number):
        if not (1 <= class_number <= 5): raise Exception("class number should be in the range of 1 to 5")
        self._class_number = class_number
```

Share Edit Follow Flag

edited Oct 16, 2022 at 13:39

answered Oct 3, 2022 at 9:38

 **Masoud Gheisari**
1,457 1 14 23

- 1 ▲ I can understand this logic flow quite easily, thanks. – ilovewt Oct 24, 2022 at 2:54

The [documentation says](#) it's just a shortcut for creating read-only properties. So

386

```
@property
def x(self):
    return self._x
```

is equivalent to

```
def getx(self):
    return self._x
x = property(getx)
```

Share Edit Follow Flag

edited Jun 9, 2022 at 5:31

 **Mateen Ulhaq**
27.1k 21 117 152

answered Jun 26, 2013 at 20:52

 **JOHN**
26.9k 5 58 88

90 The full context (most-upvoted answer) is good, but this answer was practically useful for figuring out why someone else had used `@property` as a decorator in their class. – [Joseph](#) Apr 16, 2019 at 23:23

9 "...shortcut for creating read-only properties.". The million dollar answer! – [Saptadeep](#) Oct 27, 2021 at 18:00

1 It does not create a read-only property. It creates a "standard" getter method. This statement will still work as expected: `obj.x = 5` – [Federico Razzoli](#) Apr 17, 2022 at 11:53

4 @FedericoRazzoli: `obj._x = 5` would work. `obj.x = 5` would not work (unless you were on Python 2 using old-style classes that didn't fully support descriptors), because no setter was defined. The property itself is read-only (unless you try to modify it on the class itself, not an instance of the class), it's just that Python has no obvious support for read-only attributes (best you can do is subclass `tuple` and use the `tuple`'s storage for the values, with named properties providing friendlier access; this is what `collections.namedtuple` / `typing.NamedTuple` do for you). – [ShadowRanger](#) Aug 25, 2022 at 17:04

I hadn't try recently. But yes, it's very possible that I was using Python 2. – [Federico Razzoli](#) Aug 27, 2022 at 10:55

7 A decorator is a function that takes a function as an argument and returns a closure. The closure is a set of inner functions and free variables. The inner function is closing over the free variable and that is why it is called 'closure'. A free variable is a variable that is outside the inner function and passed into the inner via decorator.

As the name says, decorator is decorating the received function.

```
function decorator(undecorated_func):
    print("calling decorator func")
    inner():
        print("I am inside inner")
    return undecorated_func
return inner
```

this is a simple decorator function. It received "undecorated_func" and passed it to inner() as a free variable, inner() printed "I am inside inner" and returned undecorated_func. When we call decorator(undecorated_func), it is returning the inner. Here is the key, in decorators we are naming the inner function as the name of the function that we passed.

```
undecorated_function= decorator(undecorated_func)
```

now inner function is called "undecorated_func". Since inner is now named as "undecorated_func", we passed "undecorated_func" to the decorator and we returned "undecorated_func" plus printed out "I am inside inner". so this print statement decorated our "undecorated_func".

now let's define a class with a property decorator:

```
class Person:
    def __init__(self, name):
        self._name = name
    @property
    def name(self):
        return self._name
    @name.setter
    def name(self, value):
        self._name = value
```

when we decorated name() with @property(), this is what happened:

```
name=property(name) # Person.__dict__ you ll see name
```

first argument of property() is getter. this is what happened in the second decoration:

```
name=name.setter(name)
```

As I mentioned above, the decorator returns the inner function, and we name the inner function with the name of the function that we passed.

Here is an important thing to be aware of. "name" is immutable. in the first decoration we got this:

```
name=property(name)
```

in the second one we got this

```
name=name.setter(name)
```

We are not modifying name obj. In the second decoration, python sees that this is property object and it already had getter. So python creates a new "name" object, adds the "fget" from the first obj and then sets the "fset".

Share Edit Follow Flag

edited Mar 19, 2022 at 2:47

answered Nov 30, 2020 at 21:02

 **Yilmaz**
48.6k 18 209 264

Your answer has a lot of typos and syntax mistakes that prevented me from reading it. – [thanos.a](#) Mar 19, 2022 at 2:44

@thanos.a I am so sorry about it :) I did edited some typos but I do not see any syntax error – [Yilmaz](#) Mar 19, 2022 at 2:48

32 I read all the posts here and realized that we may need a real life example. Why, actually, we have @property? So, consider a Flask app where you use authentication system. You declare a model User in models.py :

```
class User(UserMixin, db.Model):
    __tablename__ = 'users'
    id = db.Column(db.Integer, primary_key=True)
    email = db.Column(db.String(64), unique=True, index=True)
    username = db.Column(db.String(64), unique=True, index=True)
    password_hash = db.Column(db.String(128))
    ...

    @property
    def password(self):
        raise AttributeError('password is not a readable attribute')
```

```

@password.setter
def password(self, password):
    self.password_hash = generate_password_hash(password)

def verify_password(self, password):
    return check_password_hash(self.password_hash, password)

```

In this code we've "hidden" attribute `password` by using `@property` which triggers `AttributeError` assertion when you try to access it directly, while we used `@property.setter` to set the actual instance variable `password_hash`.

Now in `auth/views.py` we can instantiate a `User` with:

```

...
@auth.route('/register', methods=['GET', 'POST'])
def register():
    form = RegisterForm()
    if form.validate_on_submit():
        user = User(email=form.email.data,
                    username=form.username.data,
                    password=form.password.data)
        db.session.add(user)
        db.session.commit()
    ...

```

Notice attribute `password` that comes from a registration form when a user fills the form. Password confirmation happens on the front end with `EqualTo('password', message='Passwords must match')` (in case if you are wondering, but it's a different topic related Flask forms).

I hope this example will be useful

Share Edit Follow Flag

edited Sep 10, 2019 at 15:24

answered Mar 23, 2018 at 14:47

 Leo Skhnrkv
1,673 21 28

▲ property is a class behind `@property` decorator.

11 You can always check this:

▼

```
print(property) #<class 'property'>
```

🔖 I rewrote the example from `help(property)` to show that the `@property` syntax



```

class C:
    def __init__(self):
        self._x=None

    @property
    def x(self):
        return self._x

    @x.setter
    def x(self, value):
        self._x = value

    @x.deleter
    def x(self):
        del self._x

```

```

c = C()
c.x="a"
print(c.x)

```

is functionally identical to `property()` syntax:

```

class C:
    def __init__(self):
        self._x=None

    def g(self):
        return self._x

    def s(self, v):
        self._x = v

    def d(self):
        del self._x

    prop = property(g,s,d)

c = C()
c.x="a"
print(c.x)

```

There is no difference how we use the property as you can see.

To answer the question `@property` decorator is implemented via `property` class.

So, the question is to explain the `property` class a bit. This line:

```
prop = property(g,s,d)
```

Was the initialization. We can rewrite it like this:

```
prop = property(fget=g,fset=s,fdel=d)
```

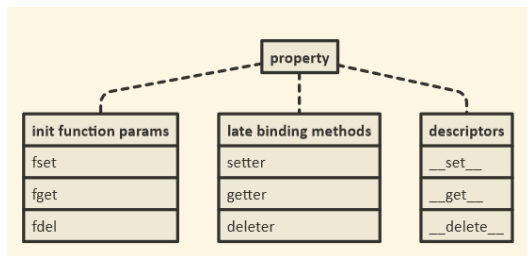
The meaning of `fget`, `fset` and `fdel`:

```

| fget
|     function to be used for getting an attribute value
| fset
|     function to be used for setting an attribute value
| fdel
|     function to be used for del'ing an attribute
| doc
|     docstring

```

The next image shows the triplets we have, from the class `property`:



`__get__`, `__set__`, and `__delete__` are there to be [overridden](#). This is the implementation of the descriptor pattern in Python.

In general, a descriptor is an object attribute with "binding behavior", one whose attribute access has been overridden by methods in the descriptor protocol.

We can also use property `setter`, `getter` and `deleter` methods to bind the function to property. Check the next example. The method `s2` of the class `C` will set the property **doubled**.

```

class C:
    def __init__(self):
        self._x=None

    def g(self):
        return self._x

    def s(self, x):
        self._x = x

    def d(self):
        del self._x

    def s2(self,x):
        self._x=x*x

    x=property(g)
    x=x.setter(s)
    x=x.deleter(d)

c = C()
c.x="a"
print(c.x) # outputs "a"

C.x=property(C.g, C.s2)
C.x=C.x.deleter(C.d)
c2 = C()
c2.x="a"
print(c2.x) # outputs "aa"

```

Share Edit Follow Flag

edited May 26, 2019 at 21:10

answered May 26, 2019 at 16:27

 **prosti**
46.1k 19 196 161

▲ This point is been cleared by many people up there but here is a direct point which I was searching. This is what I feel is important to start with the @property decorator. eg:-

23

▼

🔖

🕒

```

class UtilityMixin():
    @property
    def get_config(self):
        return "This is property"

```

The calling of function "get_config()" will work like this.

```

util = UtilityMixin()
print(util.get_config)

```

If you notice I have not used "()" brackets for calling the function. This is the basic thing which I was searching for the @property decorator. So that you can use your function just like a variable.

Share Edit Follow Flag

edited Nov 26, 2018 at 6:39

answered Nov 9, 2018 at 13:00

 **Devendra Bhat**
1,219 2 15 20

▲ Here is another example:

1

▼

🔖

🕒

```

##
## Python Properties Example
##
class GetterSetterExample( object ):
    ## Set the default value for x ( we reference it using self.x, set a value
    ## using self.x = value )
    __x = None

    ##
    ## On Class Initialization - do something... if we want..
    ##
    def __init__( self ):
        ## Set a value to __x through the getter / setter... Since __x is defined
        ## above, this doesn't need to be set...
        self.x = 1234

    return None

    ##
    ## Define x as a property, ie a getter - All getters should have a default value
    ## arg, so I added it - it will not be passed in when setting a value, so you need
    ## to set the default here so it will be used..
    ##
    @property
    def x( self, _default = None ):
        ## I added an optional default value argument as all getters should have this
        ## - set it to the default value you want to return...
        _value = ( self.__x, _default )[ self.__x == None ]

        ## Debugging - so you can see the order the calls are made...
        print( '[ Test Class ] Get x = ' + str( _value ) )

        ## Return the value - we are a getter afterall...
        return _value

    ##
    ## Define the setter function for x...
    ##
    @x.setter
    def x( self, _value = None ):
        ## Debugging - so you can see the order the calls are made...

```

```

print( '[ Test Class ] Set x = ' + str( _value ) )

## This is to show the setter function works.... If the value is above 0, set
it to a negative value... otherwise keep it as is ( 0 is the only non-negative
number, it can't be negative or positive anyway )
if ( _value > 0 ):
    self.__x = -_value
else:
    self.__x = _value

##
## Define the deleter function for x...
##
@x.deleter
def x( self ):
    ## Unload the assignment / data for x
    if ( self.__x != None ):
        del self.__x

##
## To String / Output Function for the class - this will show the property value
for each property we add...
##
def __str__( self ):
    ## Output the x property data...
    print( '[ x ] ' + str( self.x ) )

## Return a new line - technically we should return a string so it can be
printed where we want it, instead of printed early if _data = str( C( ) ) is
used....
return '\n'

##
##
##
_test = GetterSetterExample( )
print( _test )

## For some reason the deleter isn't being called...
del _test.x

```

Basically, the same as the C(object) example except I'm using **x instead...** I also don't initialize in **__init** - ... well.. I do, but it can be removed because **_x** is defined as part of the class....

The output is:

```

[ Test Class ] Set x = 1234
[ Test Class ] Get x = -1234
[ x ] -1234

```

and if I comment out the **self.x = 1234** in **__init** then the output is:

```

[ Test Class ] Get x = None
[ x ] None

```

and if I set the **_default = None** to **_default = 0** in the getter function (as all getters should have a default value but it isn't passed in by the property values from what I've seen so you can define it here, and it actually isn't bad because you can define the default once and use it everywhere) ie: **def x(self, _default = 0)**:

```

[ Test Class ] Get x = 0
[ x ] 0

```

Note: The getter logic is there just to have the value be manipulated by it to ensure it is manipulated by it - the same for the print statements...

Note: I'm used to Lua and being able to dynamically create 10+ helpers when I call a single function and I made something similar for Python without using properties and it works to a degree, but, even though the functions are being created before being used, there are still issues at times with them being called prior to being created which is strange as it isn't coded that way... I prefer the flexibility of Lua meta-tables and the fact I can use actual setters / getters instead of essentially directly accessing a variable... I do like how quickly some things can be built with Python though - for instance gui programs. although one I am designing may not be possible without a lot of additional libraries - if I code it in AutoHotkey I can directly access the dll calls I need, and the same can be done in Java, C#, C++, and more - maybe I haven't found the right thing yet but for that project I may switch from Python..

Note: The code output in this forum is broken - I had to add spaces to the first part of the code for it to work - when copy / pasting ensure you convert all spaces to tabs.... I use tabs for Python because in a file which is 10,000 lines the filesize can be 512KB to 1MB with spaces and 100 to 200KB with tabs which equates to a massive difference for file size, and reduction in processing time...

Tabs can also be adjusted per user - so if you prefer 2 spaces width, 4, 8 or whatever you can do it meaning it is thoughtful for developers with eye-sight deficits.

Note: All of the functions defined in the class aren't indented properly because of a bug in the forum software - ensure you indent it if you copy / paste

Share Edit Follow Flag

answered Aug 7, 2018 at 11:08



Acecool
754 9 13



A property can be declared in two ways.

3

- Creating the getter, setter methods for an attribute and then passing these as argument to **property** function
- Using the **@property** decorator.



You can have a look at few examples I have written about [properties in python](#).



Share Edit Follow Flag

answered Jul 13, 2017 at 9:20



nvd
487 4 2

can you updated your answer saying that property is a class so I can upvote. – [prosti](#) May 26, 2019 at 21:43



This following:

73

```

class C(object):
    def __init__(self):
        self._x = None

    @property
    def x(self):
        """I'm the 'x' property."""
        return self._x

    @x.setter
    def x(self, value):
        self._x = value

    @x.deleter

```

```
def x(self):
    del self._x
```

Is the same as:

```
class C(object):
    def __init__(self):
        self._x = None

    def _x_get(self):
        return self._x

    def _x_set(self, value):
        self._x = value

    def _x_del(self):
        del self._x

    x = property(_x_get, _x_set, _x_del,
                "I'm the 'x' property.")
```

Is the same as:

```
class C(object):
    def __init__(self):
        self._x = None

    def _x_get(self):
        return self._x

    def _x_set(self, value):
        self._x = value

    def _x_del(self):
        del self._x

    x = property(_x_get, doc="I'm the 'x' property.")
    x = x.setter(_x_set)
    x = x.deleter(_x_del)
```

Is the same as:

```
class C(object):
    def __init__(self):
        self._x = None

    def _x_get(self):
        return self._x
    x = property(_x_get, doc="I'm the 'x' property.")

    def _x_set(self, value):
        self._x = value
    x = x.setter(_x_set)

    def _x_del(self):
        del self._x
    x = x.deleter(_x_del)
```

Which is the same as :

```
class C(object):
    def __init__(self):
        self._x = None

    @property
    def x(self):
        """I'm the 'x' property."""
        return self._x

    @x.setter
    def x(self, value):
        self._x = value

    @x.deleter
    def x(self):
        del self._x
```

Share Edit Follow Flag

answered May 24, 2017 at 18:38

 **Bimo**
6,567 5 44 73

6  The first and last code examples are the same (verbatim). – [Adomas Baliuka](#) Aug 15, 2019 at 2:13 

6  I think it's intentional. Either way, this was the most useful example to me because I can grok meaning from these examples. Thanks @Bill Moore – [Jérémie](#) Jun 14, 2020 at 16:20 

 I really like this explanation the best, the other answers lose you in the example and use cases, this shows the reader exactly what is going, exactly what you get (though not comprehensive, it makes the point). – [user3481644](#) Oct 16 at 13:04

 Here is a minimal example of how `@property` can be implemented:

187

```
class Thing:
    def __init__(self, my_word):
        self._word = my_word

    @property
    def word(self):
        return self._word

>>> print( Thing('ok').word )
'ok'
```

Otherwise `word` remains a method instead of a property.

```
class Thing:
    def __init__(self, my_word):
        self._word = my_word
    def word(self):
        return self._word

>>> print( Thing('ok').word() )
'ok'
```

Share Edit Follow Flag

answered Feb 15, 2017 at 0:46

 **Alex**
12.9k 8 69 77

2  How would this example look if the `word()` function/property needed to be defined in `__init__`? – [JJ](#) Apr 9, 2017 at 9:51 

20  Can someone please explain why I would create a property decorator here, instead of just having `self._word = my_word` -- which would then work the same way `print( Thing('ok').word ) = 'ok'` – [Silver](#) Jun 9, 2017 at 4:41

3  @SilverSlash This is just a simple example, a real use-case would involve a more complicated method – [Alex](#) Jun 9, 2017 at 16:22

3 can you please explain me, how printing `Thing('ok').word` calls the function internally at runtime? – [Vicrobot](#) Dec 27, 2019 at 13:35

@property hides the fact that it's a function? Without it you would just need parenthesis to call it? – [Michael](#) Jun 23, 2022 at 23:30

|

▲ The first part is simple:

98 `@property`
`def x(self): ...`

▼ is the same as

🔖
🕒 `def x(self): ...`
`x = property(x)`

- which, in turn, is the simplified syntax for creating a `property` with just a getter.

The next step would be to extend this property with a setter and a deleter. And this happens with the appropriate methods:

```
@x.setter
def x(self, value): ...
```

returns a new property which inherits everything from the old `x` plus the given setter.

`x.deleter` works the same way.

Share Edit Follow Flag

answered Jun 26, 2013 at 20:53



[glggl](#)
90.9k 13 154 228

▲ This post is hidden. It was [deleted](#) last year by the post author.

0 I still do not understand something. Based on the definition of decorator, the following code

▼ `@name.setter def name(self,value): self._name=value`

🔖 should be equivalent to

`def name(self,value): # Line 1 self._name=value # Line 2 name = name.setter(name) # Line 3`

But after I run Lines 1-2 above, then invoking "name.setter" on Line 3 should throw an error, since at that point, "name" is no longer a property, but the plain function defined on Line 1, so it does not have the attribute "setter".

Share Edit Follow **Undelete** Flag

edited Nov 12, 2023 at 0:00

answered Nov 11, 2023 at 23:58



[István Maricza](#)
1 1

Comments disabled on deleted / locked posts / reviews

▲ This post is hidden. It was [deleted](#) last year by [Jesper](#), [Bohemian](#) ♦.

-1 ERROR! Traceback (most recent call last): File "", line 1, in ModuleNotFoundError: No module named 'property'

▼ What is with this?

🔖 Share Edit Follow **Undelete** Flag

answered Jun 26, 2023 at 6:29



[bauderr](#)
87 1 10

▲ If you have a new question, please ask it by clicking the [Ask Question](#) button. Include a link to this question if it helps provide context. - [From Review](#) – [S.B](#) Jun 26, 2023 at 7:09

Comments disabled on deleted / locked posts / reviews

▲ This post is hidden. It was [deleted](#) 2 years ago by the post author.

-2 I have another question.

▼ How '@decoration' actually works?

🔖 I just knew like this:

🕒 `@deco`
`def func():...`

`# same`
`func = deco(func)`

But it doesn't work well in '@class.method' Below is an example:

```
class Person:
    def __init__(self,name):
        self._name=name

    @property
    def name(self):
        return self._name

    @name.setter
    def name(self,value):
        self._name=value

class Person:
    def __init__(self,name):
        self._name=name

    ## @property
    def name(self):
        return self._name
    name=property(name)

    ## @name.setter
    def name(self,value):
```

```
self._name=value
name=name.setter(name) <----- error
```

The error here is because name has been changed to point to "def name(self,value):..."

I'd like to know how '@property.method' actually works.

I just guessed like that:

```
class Person:
    def __init__(self,name):
        self._name=name

    ## @property
    def name(self):
        return self._name
    name=property(name)

    ## @name.setter
    x = name.setter
    def name(self,value):
        self._name=value
    name=x.setter(name)
    del x
```

Is it right or is there other way to work that I don't know?

Thank you for reading!

Share Edit Follow **Undelete** Flag

answered Jan 31, 2022 at 5:20

 **KayyoungH L**
31 1

⬆ This does not provide an answer to the question. You can [search for similar questions](#), or refer to the related and linked questions on the right-hand side of the page to find an answer. If you have a related but different question, [ask a new question](#), and include a link to this one to help provide context. See: [Ask questions, get answers, no distractions](#) – [Marco Bonelli](#) Jan 31, 2022 at 5:30

Comments disabled on deleted / locked posts / reviews



15



🔒 This post is hidden. It was [deleted](#) last year by [user229044](#) ♦.

This answer was marked as spam or rude or abusive and is therefore not shown - you can see [the revision history](#) for details.

Share Edit Follow **Undelete** Flag

edited Feb 25, 2020 at 3:50

answered Feb 22, 2020 at 1:28

 **Victor Wang**
935 14 28

2 ⬆ This is the best source as of now – [thanos.a](#) Mar 19, 2022 at 3:00

Comments disabled on deleted / locked posts / reviews



-3



🔒 This post is hidden. It was [deleted](#) 4 years ago by [Reblochon Masque](#), [Ortomala Lokni](#), [Mark Rotteveel](#).

One remark: for me, for Python 2.x, @property didn't work as advertised when I didn't inherit from object :

```
class A():
    pass
```

but worked when:

```
class A(object):
    pass
```

for Python 3, worked always.

Share Edit Follow **Undelete** Flag

edited May 12, 2019 at 8:14

answered Dec 9, 2016 at 9:42

 **sanyassh**
8,490 15 42 80

 **Gyula Sámuel Karli**
3,302 2 16 18

5 ⬆ That's because in Python 2, a class that doesn't inherit from object is an old-style class, and old-style classes don't support the descriptor protocol (which is what property implements to work the way it does). In Python 3, old-style classes don't exist any more; all classes are what we called new-style classes in Python 2. – [chepner](#) Feb 2, 2019 at 21:39

Comments disabled on deleted / locked posts / reviews



-1



🔒 This post is hidden. It was [deleted](#) 7 years ago by the post author.

@property is a decorator which convert a method to a property(also called attribute).

Share Edit Follow **Undelete** Flag

answered Nov 5, 2017 at 4:45

 **Wizard**
21.9k 21 92 157

Comments disabled on deleted / locked posts / reviews



0



🔒 This answer is hidden. This answer was [deleted](#) via review 9 years ago by [Brian Rogers](#), [Mridang Agarwalla](#), [davidism](#), [Nekto](#), [Bahman Aries](#), [Tunaki](#).

Python 2 requires the class inherit from object, python 3 does not.

w/Python 2 a class not based on object, at member assignment, the setter method is ignored and/or replaced with a new member attribute. Which leads to unexpected behavior.

Share Edit Follow **Undelete** Flag

answered Aug 17, 2015 at 12:16

 **shmengie**
63 6

⬆ This does not provide an answer to the question. To critique or request clarification from an author, leave a comment below their post - you can always comment on your own posts, and once you have sufficient [reputation](#) you will be able to [comment on any post](#). – [Nekto](#) Aug 30, 2015 at 5:27

Comments disabled on deleted / locked posts / reviews



-1

🔒 This answer is hidden. This answer was [deleted](#) via review 10 years ago by [Paresh Mayani](#), [Soner Gönül](#), [zx8754](#), [Elias Van Ootegem](#), [greg-449](#), [julien](#).



this link came in handy for me: <http://www.programiz.com/python-programming/property>



Share Edit Follow **Undelete** Flag



answered Aug 19, 2014 at 8:59



Shankar ARUL

13.7k 13 72 73



While this link may answer the question, it is better to include the essential parts of the answer here and provide the link for reference. Link-only answers can become invalid if the linked page changes. – [greg-449](#) Aug 19, 2014 at 10:36

Comments disabled on deleted / locked posts / reviews